MACHINE LEARNING

REPORT: Multiclass Classification

VehicleVision – Sedan / SUV / Sports Car

GitHub: github.com/ParthRawat1302/Machine-learning/tree/main/Vehicle-Classification

Data: `vehicle_train_10000.csv` · `vehicle_test.csv`

1. Introduction

The goal of this project is to build a robust machine learning pipeline that predicts the type of a vehicle, `vehicle_class`, from 20 numeric technical specifications such as engine displacement, weight, braking distance and battery capacity. Every vehicle belongs to one of three classes: *Sedan* (0), *SUV* (1) or *Sports Car* (2). Submissions are scored on accuracy.

The brief added two deliberate complications that shaped every decision in this report:

- the training data contains **duplicate vehicle measurements**, and `vehicle_id` is a record identifier, not a vehicle characteristic;
- the final evaluation samples were recorded in a **different measurement environment** with additional random error, so the model that looks best on clean training data is not necessarily the best on the evaluation data.

This project includes:

- Comprehensive EDA, including a measurement of the noise in the test file
- Preprocessing pipeline and noise-aware training
- Basic models (logistic regression, decision tree, Naive Bayes) → tuned → advanced models (discriminant analysis, SVM, bagging, random forests, boosting, neural networks, pretrained transformers, ensembles)
- Feature engineering and alternative preprocessing experiments
- A noise-robust, paired cross-validation protocol with confirmation on fresh folds
- Experiments with a reduced dataset (20% sampling)
- Unsupervised learning (clustering) analysis
- Kaggle submission generation

2. Dataset Overview

We were provided with three files:

1. `vehicle_train_10000.csv` – 10,000 rows: `vehicle_id`, 20 features and the target `vehicle_class`
2. `vehicle_test.csv` – 2,500 rows with the same 20 features, no target

3. `vehicle_sample_submission.csv` – the required format (`vehicle_id`, `vehicle_class`)

2.1 Target Variable

`vehicle_class` has three values. After removing duplicate rows (Section 3.1) the training set holds 7,500 distinct vehicles:

Class	Label	Vehicles	Share
0	Sedan	2,539	33.85%
1	SUV	2,490	33.20%
2	Sports Car	2,471	32.95%

Table 1. Class distribution after de-duplication.

The classes are almost perfectly balanced, so **accuracy is a fair metric**: a model that always predicts the largest class scores only 33.85%.

2.2 Input Features

All 20 features are numeric. They fall into four physical groups:

- **Engine & power** – displacement, horsepower, torque, cylinders, tire width
- **Size & weight** – weight, wheelbase, length, width, height, fuel tank, cargo capacity, ground clearance, turning radius
- **Performance & efficiency** – city and highway km/l, 0–100 km/h time, braking distance, drag coefficient
- **Electrical** – battery capacity

3. Exploratory Data Analysis (EDA)

3.1 Data Quality

- **No missing values** in either file, and every column is numeric.
- **2,500 of the 10,000 training rows (25%) are exact duplicates** – every measurement *and* the label repeated. No duplicated measurement ever carries a different label, so removing them loses no information.
- `vehicle_id` is unique and does not overlap between train and test.

3.2 Which Features Separate the Classes

We ranked the features with a one-way ANOVA F statistic (between-class over within-class variance). The strongest single discriminators are `battery_capacity_kwh` (F = 6,239), `braking_distance_m` (F = 6,073), `acceleration_0_100_kmh_s` (F = 5,165). Figure 1 shows how the classes overlap on each of them: battery capacity and acceleration isolate sports cars, braking distance separates SUVs, and sedans sit in the middle.

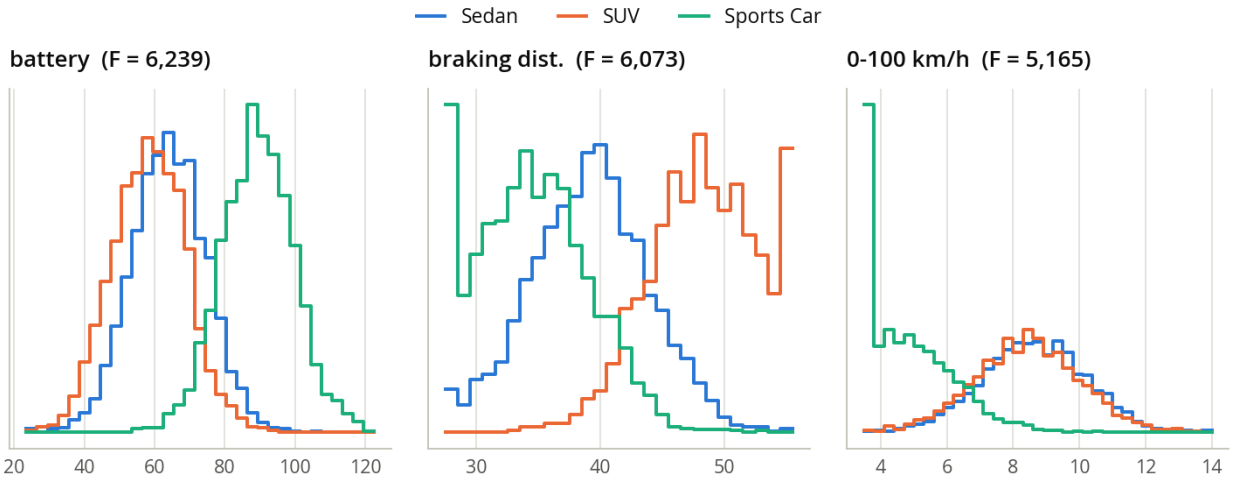


Figure 1. Class-wise distributions of the three most discriminative features.

Feature	Unit	ANOVA F	Rank	% rows on a clip limit
battery_capacity_kwh	kWh	6,239	1	0.0%
braking_distance_m	m	6,073	2	7.0%
acceleration_0_100_kmh_s	s	5,165	3	7.1%
vehicle_width_mm	mm	5,068	4	11.6%
wheelbase_mm	mm	4,909	5	0.5%
vehicle_length_mm	mm	4,529	6	3.9%
vehicle_weight_kg	kg	4,271	7	3.0%
drag_coefficient	-	3,754	8	2.2%
fuel_tank_capacity_l	litres	3,690	9	7.1%
horsepower_hp	hp	2,648	10	1.5%
torque_nm	Nm	2,638	11	4.6%
cargo_capacity_l	litres	2,145	12	13.2%
engine_displacement_l	litres	2,075	13	0.6%
engine_cylinders	count	2,043	14	3.9%
tire_width_mm	mm	1,694	15	6.9%
vehicle_height_mm	mm	1,663	16	3.3%
ground_clearance_mm	mm	1,547	17	26.2%
turning_radius_m	m	1,081	18	0.0%
city_fuel_efficiency_kmpl	km/l	565	19	4.1%
highway_fuel_efficiency_kmpl	km/l	324	20	0.3%

Table 2. All 20 features ranked by class-separating power, with the share of rows clipped to a fixed limit (Section 3.5).

3.3 Correlation Structure

The features are **strongly correlated**: 90 of the 190 feature pairs have |r| > 0.5, and the largest are `horsepower_hp` ↔ `torque_nm` (0.959); `engine_displacement_l` ↔ `torque_nm` (0.952); `wheelbase_mm` ↔ `vehicle_width_mm` (0.952); `engine_displacement_l` ↔ `horsepower_hp` (0.950). Figure 2 shows two large blocks: a *size* block (length, wheelbase, weight, width, fuel tank) and a *power* block (displacement, torque, horsepower, tire width).

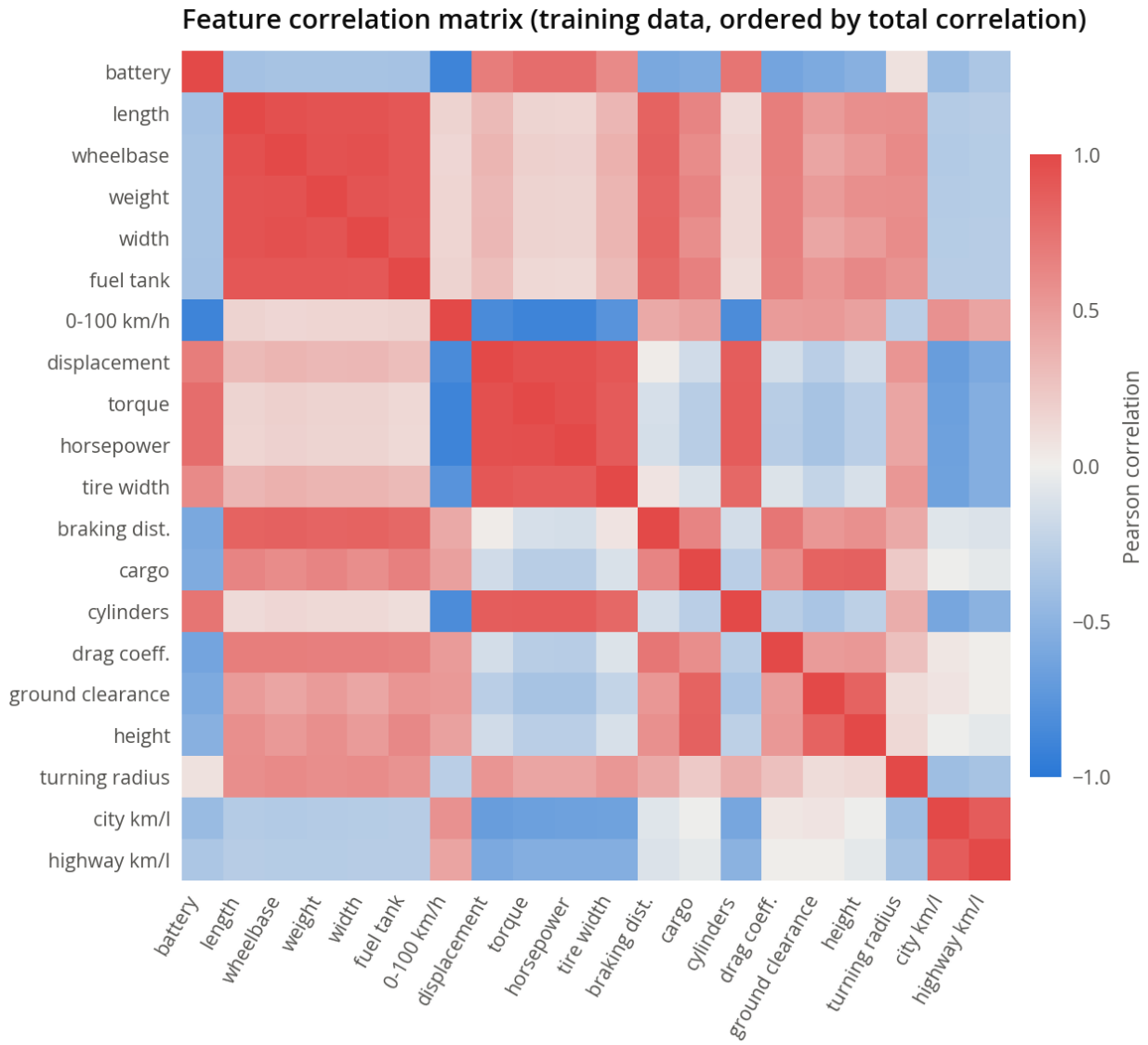


Figure 2. Pearson correlation matrix of the training features (blue = negative, red = positive).

This matters most for Naive Bayes, whose core assumption is that features are independent given the class. With correlations near 0.95 that assumption is badly violated (Section 7.3).

3.4 PCA Visualisation

Two principal components explain 41% and 38% of the variance (79% together), matching the size and power blocks. The covariance is ill-conditioned (largest/smallest eigenvalue = 265). Figure 3 shows that the classes form overlapping clouds: SUVs on the right, sports cars upper-left, and **sedans in the middle, overlapping both**. The problem is close to linearly separable but not perfectly, so some error is unavoidable.

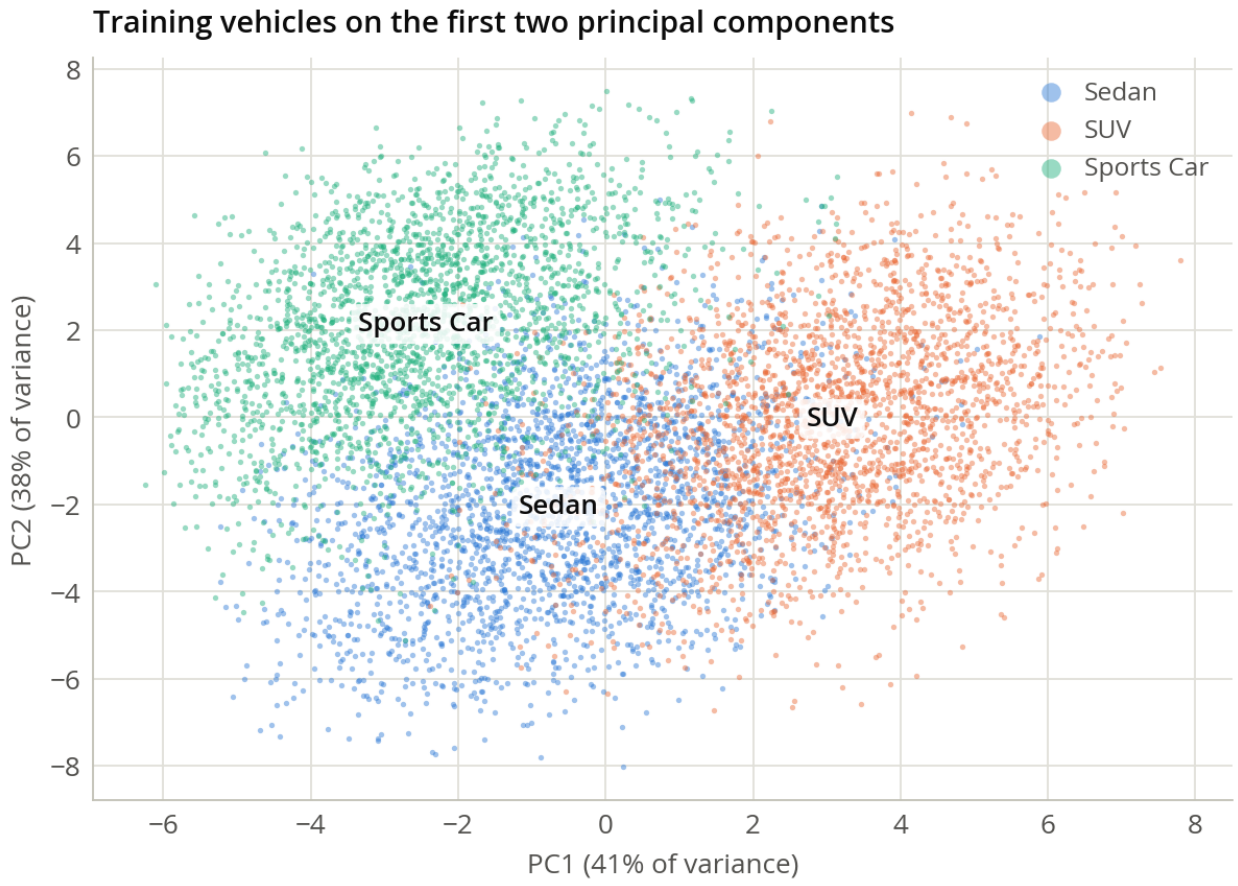


Figure 3. The 7,500 training vehicles projected on the first two principal components.

3.5 Clipped Measurements

Many features are clipped to fixed limits, creating point masses: 23.1% of vehicles sit exactly at a ground clearance of 110 mm, 12.6% at a cargo capacity of 150 l, 7.1% at a 0–100 km/h time of 3.5 s and 11.6% at the width limits (1,650 / 2,050 mm). The test file is clipped at exactly the same limits.

3.6 Train vs Test: Measuring the Extra Noise

Because the brief warns that the evaluation data is noisier, we measured the noise directly instead of guessing it:

- Means and standard deviations barely move (mean test/train SD ratio 1.009), so a simple drift check cannot see the noise.
- Measurement error **weakens correlations**. Across the strongly correlated pairs, test correlations are on average 0.9932 of the training ones – outside the 0.9947–1.0048 range that random 2,500-row samples of the training data produce. Adding simulated noise to the training data reproduces this only at about **0.08 standard deviations** (Figure 4, left).
- The clip spikes **shrink** in the test file (ground clearance: 23.08% → 17.40%), and the lost mass reappears just inside the limit. So the noise was added *after* clipping, and it reaches roughly half of the values (Figure 4, right).
- **Adversarial validation** (a classifier trained to tell train rows from test rows) reaches AUC 0.559 ± 0.006, against 0.504 for two random halves of the training set. The only separating signals are this noise signature; no feature has shifted on its own.
- There are **no exact or near duplicates** between train and test (the closest test vehicle is 0.886 standardised units from any training vehicle), the predicted class mix of the test set matches training (χ^2 p = 0.95), and no digit or rounding pattern leaks the class.

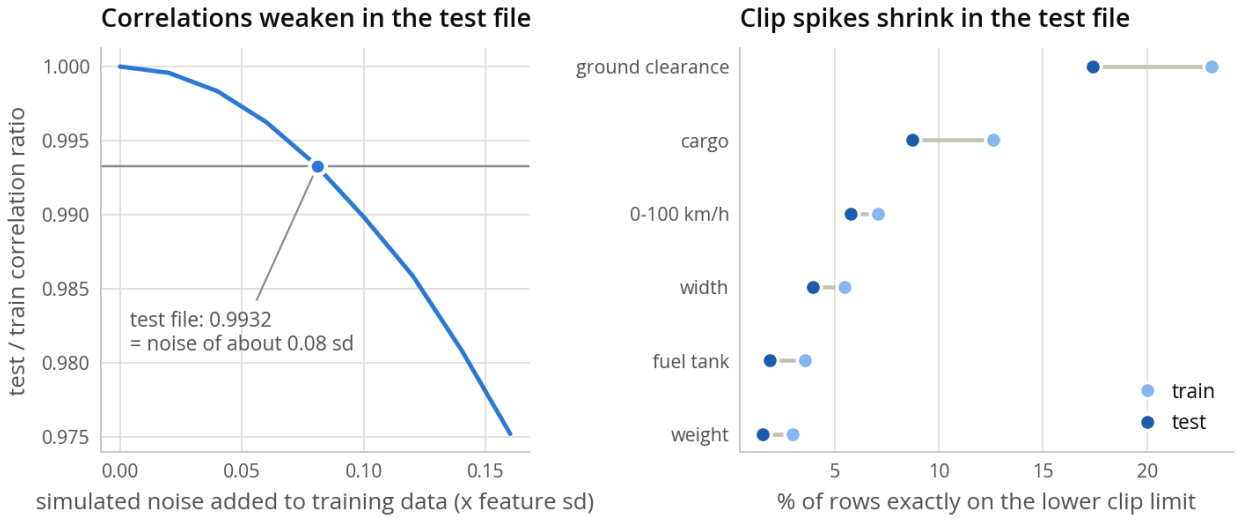


Figure 4. Two independent measurements of the extra noise in the test file.

Conclusion of the EDA: the data is clean apart from the duplicates, highly correlated, nearly balanced and close to linearly separable, and the test file carries mild measurement noise (about 0.08 SD on average, after clipping). Every model in this report is therefore evaluated *under that noise*, not on clean data.

4. Preprocessing Pipeline

4.1 Removing Duplicates

The 2,500 duplicate rows were dropped **before any split**. If they stayed, a vehicle and its copy could land on opposite sides of a cross-validation split, so the model would be validated on rows it memorised. We measured the effect with plain 5-fold CV:

Model	With duplicates	Without	Inflation
Gaussian Naive Bayes	84.39	84.65	−0.26
1-nearest-neighbour	89.06	82.97	+6.09
Decision tree (unlimited depth)	89.61	82.29	+7.32

Table 3. Cross-validated accuracy (%) with and without the duplicate rows.

Models that can memorise look up to **7 points better than they really are**, while Naive Bayes (which stores only means and variances) is unaffected. The bias depends on the model, so it would corrupt every comparison between models.

4.2 Dropping the Identifier

`vehicle_id` identifies a record, not a vehicle property; the brief says so explicitly, and the IDs do not overlap between train and test. It is excluded from every model.

4.3 Missing Values and Encoding

No values are missing and all features are numeric, so no imputation or categorical encoding was needed. The pipeline is still built so that any such step would be fitted inside the training folds.

4.4 Feature Scaling

- `StandardScaler` for every scale-sensitive model (logistic regression, LDA, SVM, neural networks). The features range from litres (~2) to millimetres (~4,600); without scaling, a single L2 penalty would effectively switch off the large-unit features.
- Tree models (decision trees, forests, boosting) use the raw values, since splits are unaffected by scaling.

4.5 Noise-Aware Training (Data Augmentation)

Because the evaluation data is noisier, the final model is trained on the 7,500 clean rows **plus four jittered copies** of them, each with Gaussian error of 0.20 standard deviations per feature (37,500 rows in total). This teaches the model that a vehicle keeps its class when its measurements wobble. Variants that copy the test noise exactly (re-clipped, re-rounded, partial) were tested in Section 5.

4.6 Pipeline Construction

Each model is a `Pipeline` (`StandardScaler` → model). The scaler, any projection (PCA, LDA) and any augmentation are fitted on the training fold only, which guarantees identical processing for training, validation and test data and prevents leakage.

5. Feature Engineering and Alternative Preprocessing

On top of the base pipeline we tried a wide range of feature engineering and preprocessing ideas in front of logistic regression, our best model (Section 7.1), and scored each one with the validation protocol of Section 6 (Table 4):

- **New features:** physically meaningful ratios, polynomial interactions, spline expansions and clip-limit flags.
- **Transformations:** Yeo-Johnson and rank-Gauss transforms to make skewed features more Gaussian, and robust scaling.
- **Un-censoring:** each value sitting exactly on a clip limit was replaced by the expected true value beyond the limit, predicted from the other 19 features (a truncated-normal mean). This touched 45% of the rows.
- **Denoising and selection:** dropping small principal components, ANOVA feature selection, lasso, and removing training rows the model itself finds implausible.
- **Noise-aware training** with 5–10 copies that reproduce the test noise exactly.

Experiment	Test-like	Screening gap	Fresh-fold re-test
10 physics ratios (power/weight, hp per litre, …)	89.24	−0.02	–
Degree-2 polynomial features (230)	89.20	−0.06	−0.01
Spline expansion (GAM, 5 knots)	89.12	−0.15	–
Clip-limit indicator flags	89.21	−0.06	–
Yeo-Johnson power transform	89.32	+0.06	0.00
Rank-Gauss (quantile) transform	89.27	+0.01	–
Robust scaling (median / IQR)	89.27	+0.01	–
Un-censoring clipped values	89.28	+0.02	+0.01
PCA denoising (17 components)	89.23	−0.04	–
Feature selection (top 17 by ANOVA)	88.75	−0.52	–
Lasso (L1) logistic regression	89.25	−0.02	–
Data cleaning (drop implausible rows)	89.20	−0.07	–
Training on 10 test-noise copies	89.21	−0.05	–

Table 4. Feature engineering and preprocessing experiments (logistic regression).

Result: none of them improves the model. The Yeo-Johnson transform is a useful lesson: it looked like a win in screening (+0.06 points, about three standard errors) but scored 0.00 on

fresh folds – the *winner's curse* that the fresh-fold confirmation (Section 6) exists to catch. The standardised raw features are already the right representation for a linear boundary.

6. Validation Strategy

A single 80/20 split is too noisy for this problem: at ~89% accuracy on 1,500 validation rows the standard error is about 0.8 points, larger than most differences between models. We therefore used:

- **Repeated stratified 5-fold cross-validation** – 3 repeats = 15 folds of 6,000 training / 1,500 validation rows.
- **Test-like scoring** – each validation fold is corrupted with the noise measured in Section 3.6, in its two readings (0.08 SD on every value, and 0.12 SD on a random half of the values), re-clipped and re-rounded exactly like the test file. *Test-like accuracy* is the mean of the two; a 0.15 SD *stress* level is also reported.
- **Paired comparisons** – every model sees the same folds and the same noisy copies, so each difference against the final model comes with its own standard error (shown as ±).
- **Confirmation on fresh folds** – comparing dozens of configurations inflates the best score by luck (the *winner's curse*). Any model that beat logistic regression in screening was re-tested on 15 new folds (3 new seeds) and only accepted if it still won by more than two standard errors.

7. Basic Models

We started with the three basic models required for the project, from a simple baseline to a tuned version of each. Table 5 summarises them; all numbers are accuracy (%) on the 15 paired folds.

Model	Clean	Test-like	Stress 0.15	vs final
Logistic regression – baseline (C = 1)	89.44	89.20	88.92	−0.07 ± 0.05
Logistic regression – tuned (C = 0.1, noise aug.)	**89.58**	**89.26**	**88.96**	**–**
Decision tree – depth 6	86.16	86.05	85.65	−3.21 ± 0.12
Decision tree – depth 10	85.22	85.12	84.68	−4.14 ± 0.15
Decision tree – depth 4 + LDA features	87.39	87.21	86.82	−2.05 ± 0.14
Gaussian Naive Bayes (raw features)	84.67	84.64	84.53	−4.62 ± 0.19
Naive Bayes + PCA whitening (10 comps)	86.26	86.12	85.86	−3.14 ± 0.14
Naive Bayes + LDA(2) projection + aug.	89.36	89.20	88.84	−0.07 ± 0.04

Table 5. Basic models. Clean = no added noise; Test-like = calibrated test noise (the selection metric); vs final = paired gap to the final model ± one standard error.

7.1 Logistic Regression (Baseline → Tuned)

- Baseline (scaled, C = 1): 89.20% test-like (89.44% clean).
- Tuned: the inverse regularisation strength C was swept from 0.01 to 1,000. Accuracy is flat from C = 0.01 to 1 (within ±0.05 points); larger C values were slightly worse under noise. **C = 0.1 with noise augmentation gives 89.26% test-like (89.58% clean)** – the best model of the whole project.
- Multinomial (softmax) training beats one-vs-rest (88.36%), and the log-loss beats a hinge loss (linear SVM, 88.07%) and least squares (ridge classifier, 86.38%).
- Why it wins: the class boundary is essentially linear (Section 3.4), and a linear model with an L2 penalty cannot over-fit the tiny, noise-sensitive directions of an ill-conditioned covariance.

7.2 Decision Tree

- Depth 6: 86.05% test-like; depth 10: 85.12% (over-fits).
- A tree splits on one feature at a time, so it approximates the tilted linear boundary with a staircase. Giving it the two LDA discriminant directions as extra features lifts it to 87.21% at depth 4.
- Trees also degrade fastest under measurement noise (Section 9): a small error can push a value across a split threshold.

7.3 Naive Bayes

- Gaussian Naive Bayes on the raw features: **84.64%** – almost five points below the other models. The cause is the independence assumption: with correlations near 0.95 the model multiplies the same evidence several times.
- Scaling cannot help (Gaussian NB fits a mean and variance per feature, which absorbs any linear rescaling), and making each feature more Gaussian (power or quantile transforms) did not help either.
- **Fixing the coordinate system works**: PCA whitening reaches 86.12%, and projecting onto the two LDA discriminant directions reaches **89.20%**, statistically tied with logistic regression (−0.02 ± 0.04 on fresh folds). The LDA projection is fitted inside each training fold, so there is no leakage.

8. Advanced Models

We then tried every major family of advanced classifiers, including GPU-accelerated ones (NVIDIA RTX 4060 Laptop GPU, 8 GB). Table 6 lists the best configuration of each; Figure 5 compares them.

Family	Best configuration	Test-like	vs final
Discriminant analysis	LDA, covariance shrinkage 0.1	89.18	−0.09 ± 0.03
	QDA, reg_param 0.1	88.86	−0.41 ± 0.06
	Regularised DA (α = 0.1)	89.13	−0.13 ± 0.03
	Mixture DA (4 tied Gaussians / class)	88.35	−0.92 ± 0.08
Kernel methods	SVM, RBF kernel, C = 1	89.08	−0.18 ± 0.05
Bagging	Bagged decision trees (300)	87.87	−1.40 ± 0.08
	Bagged logistic regression (50)	89.24	−0.03 ± 0.02
Random forests	Random forest (500 trees)	88.27	−1.00 ± 0.08
	Random forest + LDA features + noise aug.	88.78	−0.49 ± 0.06
	Extra-trees + LDA features	88.67	−0.60 ± 0.08
Boosting	AdaBoost (depth-2 trees)	87.52	−1.74 ± 0.09
	Gradient boosting (scikit-learn)	88.44	−0.83 ± 0.10
	HistGradientBoosting	87.92	−1.35 ± 0.10
	XGBoost on GPU + LDA features	88.85	−0.42 ± 0.07
	LightGBM + LDA features + noise aug.	88.79	−0.47 ± 0.09
	CatBoost on GPU + LDA features	88.76	−0.50 ± 0.06
Neural networks	MLP 64×2, input noise 0.2, dropout 0.1	89.01	−0.25 ± 0.06
	FT-Transformer (trained from scratch)	88.63	−0.63 ± 0.09
	TabPFN v2 (pretrained transformer)*	89.12	−0.24 ± 0.08
Final model	**Logistic regression, C = 0.1 + noise aug.**	**89.26**	**–**

*Table 6. Advanced models, best configuration per family (accuracy %, 15 paired folds). *TabPFN is slow, so it was compared on the 5 folds of one repeat (logistic regression scored 89.36% on those folds).*

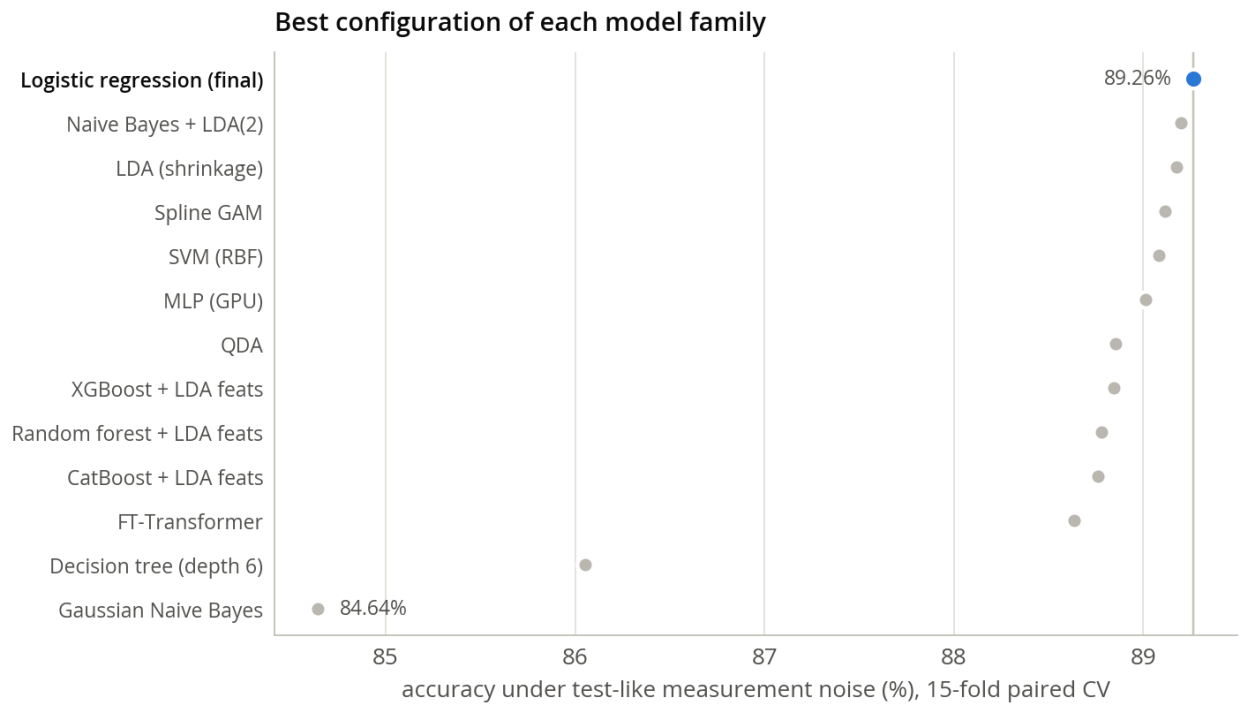


Figure 5. Accuracy under test-like noise of the best configuration in each family. Every family lands below logistic regression.

8.1 Discriminant Analysis (Bayes with Full Covariance)

LDA models a shared covariance and gives a linear boundary; it is statistically tied with logistic regression (89.18%, −0.02 ± 0.03 on fresh folds). QDA fits a covariance per class and scores lower (88.86%), even when blended with the pooled covariance (RDA) or extended to several Gaussians per class (88.35%). The classes therefore share essentially one covariance, which is exactly the situation in which the linear boundary is optimal.

8.2 SVM

An RBF-kernel SVM reaches 89.08%. A linear SVM (hinge loss) is weaker at 88.07%. The kernel adds flexibility the data does not need.

8.3 Bagging and Random Forests

Averaging trees smooths the staircase boundary: bagged trees score 87.87% and a 500-tree random forest 88.27%. Adding the LDA directions as features raises the forest to 88.78%, still clearly below the linear model. Bagging logistic regression changes almost nothing (89.24%), as expected for a stable model.

8.4 Boosting (including GPU XGBoost and CatBoost)

Boosted trees follow the same pattern: AdaBoost 87.52%, scikit-learn gradient boosting 88.44%, plain XGBoost 88.22%. With the LDA features, XGBoost reaches 88.85%, LightGBM 88.79% and CatBoost 88.76%. XGBoost and CatBoost ran on the GPU (one to four minutes per 15-fold configuration).

8.5 Neural Networks

- **Batched GPU MLPs.** Instead of training one network at a time, we stacked hundreds of independent networks into batched matrix multiplications, each with its own fold, input noise, weight decay and dropout. A grid of 54 configurations × 15 folds × 3 seeds (**2,430 networks**) trained in about 6.5 minutes.
- The trend was monotonic: **more capacity, lower accuracy**. The best MLP (64×2, input noise 0.2, dropout 0.1) scored 89.01%; 256-wide networks lost 1–3 points; every bit of regularisation helped, pushing the network towards a linear model.
- An FT-Transformer (self-attention over feature tokens) scored 88.63% (small) and 88.04% (large).
- **TabPFN v2**, a transformer pretrained to perform Bayesian inference on small tables, scored 89.12% against 89.36% for logistic regression on the same folds.

8.6 Ensembles, Stacking and Post-processing

Averaging logistic regression with a different model never helped, because the partners make the same errors plus some of their own:

Combination	Test-like	vs final
LogReg + spline GAM (trained on 5 noisy copies)	89.28	+0.01 ± 0.05
LogReg + polynomial LogReg	89.24	−0.02 ± 0.02
LogReg + LDA + Naive Bayes	89.24	−0.02 ± 0.03
LogReg + SVM	89.24	−0.03 ± 0.04
LogReg + MLP	89.23	−0.04 ± 0.05
LogReg + random forest (LDA features)	89.19	−0.07 ± 0.04
LogReg + XGBoost	89.17	−0.09 ± 0.04
LogReg + CatBoost	89.11	−0.15 ± 0.03
Stacking: meta-model over 12 models	89.07	−0.20 ± 0.06
Per-class threshold tuning (cross-fitted)	89.25	−0.01 ± 0.02

Table 7. Probability averages with the final model, stacking and threshold tuning. The best average re-tested at +0.01 on fresh folds – a tie.

9. Robustness to Measurement Noise

Because the evaluation data is noisy, robustness matters as much as clean accuracy. Figure 6 and Table 8 show accuracy as increasing noise is added to the validation rows (re-clipped and re-rounded like the test file).

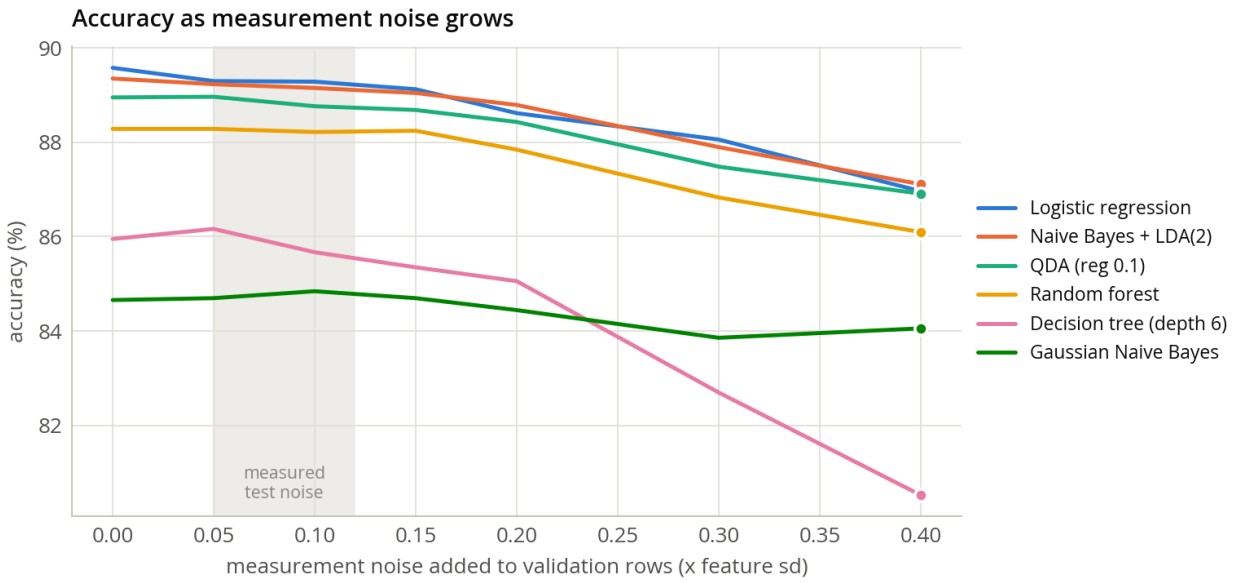


Figure 6. Accuracy versus measurement noise (5 folds of one CV repeat). The shaded band marks the noise measured in the test file.

Model	0	0.05	0.1	0.15	0.2	0.3	0.4
Logistic regression	**89.57**	**89.29**	**89.28**	**89.12**	**88.61**	**88.05**	**86.96**
Naive Bayes + LDA(2)	89.35	89.23	89.15	89.04	88.79	87.89	87.11
QDA (reg 0.1)	88.95	88.96	88.76	88.68	88.43	87.48	86.91
Random forest	88.28	88.28	88.21	88.24	87.84	86.83	86.09
Decision tree (depth 6)	85.95	86.16	85.67	85.35	85.05	82.69	80.52
Gaussian Naive Bayes	84.65	84.69	84.84	84.69	84.44	83.85	84.05

Table 8. Accuracy (%) by noise level (× feature standard deviation).

- Logistic regression and Naive Bayes + LDA stay on top at every realistic noise level and lose about 2.5 points at an extreme 0.40.
- QDA sits about 0.5 points lower at realistic noise levels and only catches up at 0.40; the random forest stays about one point lower throughout.
- The decision tree collapses fastest (−5.4 points at 0.40).
- Gaussian Naive Bayes is flat but low: its error is set by the wrong independence assumption, not by noise.

10. Final Model

Logistic regression (multinomial, L2 penalty C = 0.1, standardised features) trained on the 7,500 de-duplicated vehicles plus four noise-jittered copies (37,500 rows).

- Accuracy under test-like noise: **89.26%** on the 15 screening folds and **89.20%** on the 15 fresh confirmation folds; 89.58% on clean data.

- It beat or tied every one of the more than 100 configurations tested, and nothing passed the fresh-fold re-test against it.

True \ Predicted	Sedan	SUV	Sports Car	Precision	Recall	F1
Sedan	2,150	218	170	0.845	0.847	0.846
SUV	222	2,258	10	0.908	0.907	0.907
Sports Car	172	11	2,288	0.927	0.926	0.927

Table 9. Confusion matrix and per-class metrics of the final model (out-of-fold, test-like noise, 7,500 vehicles).

Almost all errors are Sedan ↔ SUV and Sedan ↔ Sports Car; SUVs and sports cars are almost never confused (21 of 7,500). This is the overlap visible in Figure 3. The errors sit close to the decision boundary (63% of misclassified vehicles still received 20–50% probability for the true class), so they come from genuinely overlapping classes, not from noisy labels or missing structure. That explains why no model got past about 89.3%.

11. Kaggle Submission

We generated a submission file for every model family – 51 files in total, each validated against the sample submission (2,500 rows, matching IDs and order, labels 0/1/2):

- Logistic regression (baseline, tuned, and variants)
- Naive Bayes (raw, PCA, LDA)
- LDA / QDA / mixture discriminant analysis
- SVM, decision tree ensembles and boosting
- MLP, FT-Transformer and TabPFN
- Probability-averaged ensembles

Our submissions scored **about 0.90 on the public leaderboard**. The public score uses only part of the test set, so it moves by roughly ±0.6–1.1 points from sampling luck alone; differences of a few tenths between near-identical models cannot be read from it.

12. Experiments with 20% Dataset (Full vs 20% Comparison)

To see how each model depends on the amount of data, we trained it on a stratified 20% of every training fold (1,199 vehicles instead of 6,000) and scored it on exactly the same noisy validation rows.

Model	100% data	20% data	Change
Logistic regression	89.36	88.78	−0.58
Naive Bayes + LDA(2)	89.22	89.00	−0.22
SVM (RBF, C=1)	89.09	88.52	−0.57
Neural network (MLP 64x2)	88.99	87.75	−1.24
XGBoost + LDA features	88.89	88.05	−0.84
QDA (reg 0.1)	88.77	88.51	−0.26
Random forest	88.26	87.72	−0.53
Decision tree (depth 6)	85.71	84.15	−1.56
Gaussian Naive Bayes	84.68	84.70	+0.02

Table 10. Test-like accuracy (%) with 100% and 20% of the training data (5 folds).

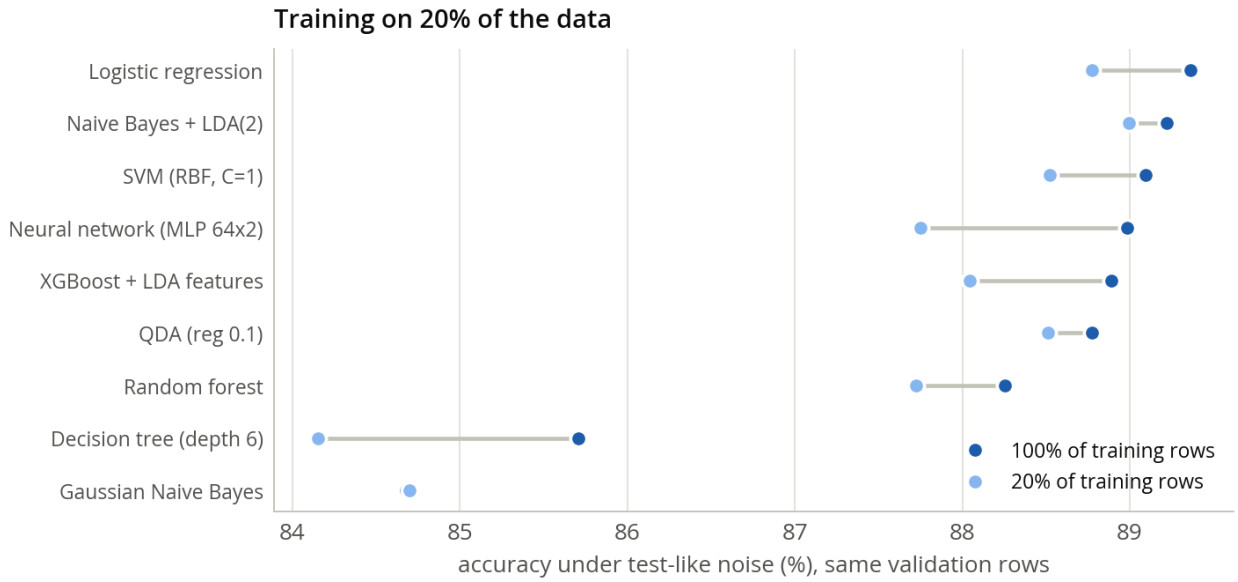


Figure 7. Change in accuracy when only 20% of the training data is used.

- Models with few parameters barely notice: Naive Bayes + LDA loses only 0.22 points and Gaussian Naive Bayes does not change.
- Logistic regression and the SVM lose about 0.6 points but stay near the top.
- Flexible models need the data most: the MLP loses 1.24 points and the decision tree 1.56.
- **Conclusion:** on this problem the simple generative and linear models are the most data-efficient, while trees and neural networks depend strongly on dataset size.

13. Unsupervised Learning

Clustering was performed on the standardised features, without labels, to see whether the three vehicle types form natural groups. We used K-Means (k = 3), a Gaussian mixture model (k = 3), agglomerative clustering (Ward linkage, k = 3) and DBSCAN (eps = 2.20, min_samples = 10). Quality was measured with the silhouette score, and agreement with the true classes with the adjusted Rand index (ARI), homogeneity, and the accuracy after optimally matching clusters to classes.

Algorithm	Clusters	Silhouette	ARI	Homogeneity	Matched acc.
K-Means (k=3)	3	0.293	0.532	0.484	81.3%
Gaussian mixture (k=3)	3	0.141	0.356	0.369	71.3%
Agglomerative (Ward, k=3)	3	0.228	0.378	0.354	71.8%
DBSCAN	1 (+58 noise)	–	0.000	0.000	33.6%

Table 11. Clustering results against the true vehicle classes.

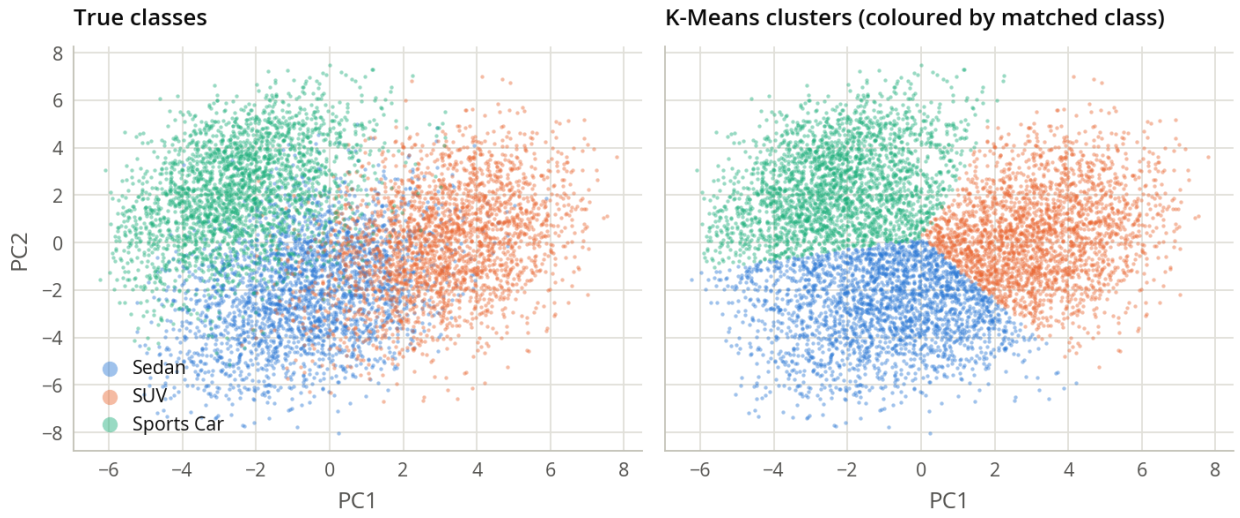


Figure 8. True classes (left) and K-Means clusters (right), each cluster coloured by the class it matches best.

- **K-Means recovers the vehicle types surprisingly well**: after matching clusters to classes it is right for 81.3% of vehicles (ARI 0.53), because the classes are roughly spherical clouds of similar size.
- The Gaussian mixture and Ward clustering split the data less cleanly (about 71%).
- DBSCAN finds a single dense cluster: the classes touch, so there are no low-density gaps for it to cut along.
- Even the best clustering is about eight points below supervised learning: the boundary between sedans and the other types is not visible without labels. Clustering is useful for exploration, not as a replacement for classification.

14. Key Insights & Discussion

1. **The boundary is linear, so logistic regression wins.** Every more flexible family (kernels, trees, boosting, neural networks, transformers) scored lower, in order of flexibility.
2. **Naive Bayes needs the right coordinate system.** On raw, highly correlated features it is almost five points behind; on the LDA projection it ties logistic regression.
3. **Removing duplicates is essential** for honest validation, especially for models that can memorise (trees, nearest neighbours).
4. **Measuring the test noise beats guessing it.** Correlation shrinkage and clip-spike shrinkage both put it at about 0.08 standard deviations, so every model was scored under realistic noise.
5. **Robustness and accuracy point the same way here:** the regularised linear model is both the most accurate and among the most noise-robust.
6. **Confirmation on fresh folds prevents false improvements:** the Yeo-Johnson transform looked three standard errors better in screening and was exactly tied on new folds.
7. **About 89.3% is the ceiling of this dataset**: the remaining error is genuine overlap between sedans and the other two types.

15. Limitations

- The noise model is estimated from the test file; the private evaluation could differ slightly.
- Public-leaderboard differences of a few tenths are mostly sampling luck and cannot confirm small improvements.
- Very slow models (TabPFN) were evaluated on 5 folds instead of 15.
- Overlap between sedans and the other classes limits every model to about 89–90% accuracy.

16. Conclusion

This project built a complete machine learning system for three-class vehicle classification. We covered the entire pipeline:

- Data cleaning (2,500 duplicates removed) and EDA
- Measurement of the test-set noise
- Preprocessing and noise-aware training
- Basic models: logistic regression, decision tree, Naive Bayes

- Advanced models: discriminant analysis, SVM, bagging, random forests, boosting, neural networks and pretrained transformers, partly on the GPU
- Feature engineering, ensembles and stacking
- 20% dataset comparison
- Unsupervised clustering
- Kaggle submission

The final model, a noise-augmented logistic regression, reached **89.26%** accuracy under test-like noise in cross-validation (89.20% on fresh folds) and about **0.90** on the public leaderboard – at the limit set by the overlap between the vehicle classes.

Appendix: Code

File	Purpose
`common.py`	Loading, de-duplication, noise simulation, CV harness, submission writing
`eda.py`	Exploratory data analysis and figures
`logistic_regression.py`	Method 1 – logistic regression (baseline → tuned)
`naive_bayes.py`	Method 2 – Naive Bayes (raw, PCA, LDA projection)
`bayes_discriminant.py`	Method 3 – LDA / QDA
`advanced_models.py`	GPU benchmark: batched MLPs, XGBoost, CatBoost, TabPFN, ensembles, fresh-fold confirmation
`make_all_submissions.py`	Writes a validated submission for every model